

LAPORAN PRAKTIKUM
Topologi Jaringan Terdesentralisasi (ESP-NOW dan Mesh)
LAB SESSION 05



Disusun Oleh:

NAMA : Fandika Nugraha
NPM : 25782097
KELAS : TRI – 3D
MATA KULIAH : Teknologi IoT II

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.
Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG
2025/2026

5.5 Praktikum

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



Secreenshot Serial Monitor: Tampilan Mac Address

```
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahId;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
```

Output: Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 ESP-12E Module' on 'COM3')

New Line 115200 baud

ESP8266 MAC Address: C8:CB:1A:3:87:46:8F

5.5.2 Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

```
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // Struktur data penerima (wajib identik dengan sisi pengirim)
5  typedef struct struct_pesan {
6      int perintahID;
7      int nilaiParameter;
8  } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antena mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[38];
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 33

Paket masuk dari: ec:fa:bc:3d:2d:a1
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 42

```
16     unsigned long previousMillis = 0;
17     const long interval = 2000; // Transmisi berkala setiap 2 detik
```

Output Serial Monitor X

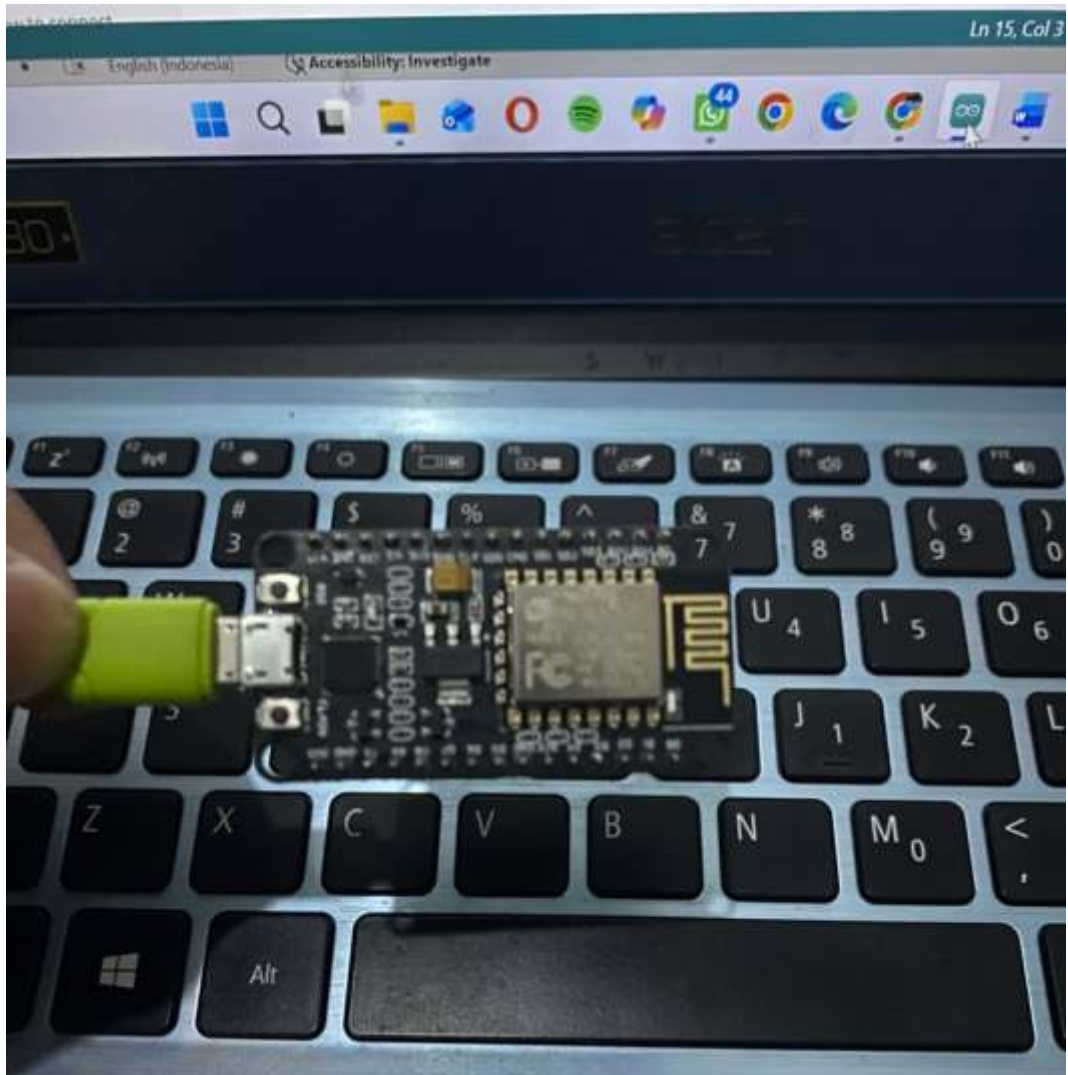
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:87:46:8f | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:87:46:8f | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:87:46:8f | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:87:46:8f | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima
Kirim paket ke: c8:c9:a3:87:46:8f | Status: Berhasil Diterima
Kirim paket ke: bc:dd:c2:2d:ed:57 | Status: Berhasil Diterima

Paket masuk dari: ec:fa:bc:3d:2d:a1
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 97

Paket masuk dari: ec:fa:bc:3d:2d:a1
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 76

5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)



```
Output  Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)')

=====
DATA TERBARU DARI NODE #2
Parameter 1 : 26.00
Parameter 2 : 76.50
=====
DATA TERBARU DARI NODE #2
Parameter 1 : 28.80
Parameter 2 : 79.40
=====
```

```
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data
20
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus)
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub
Pengiriman Node #2 -> Sukses Diterima Hub

80°F
Mostly clear



Search

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)



```
Sketchy.cpp
1 #include <rainlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "IoTMesh" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "123" // Kata sandi jaringan Mesh
7 #define MESH_PORT 8555 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 1 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 72.9]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 72.8]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 72.8]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31, "kelembapan": 72.8]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 72.8]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 72.9]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31, "kelembapan": 73]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31, "kelembapan": 73.1]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 73.9]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 73.6]
[KIRIM MESH] ["node": "Node-1", "chipId": 3158125985, "aahu": 31.1, "kelembapan": 73.1]
```

```
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

[TERIMA DARI] Node-1 (Node ID: 3158125985 | Chip ID: 3158125985)
Suhu : 30.60 °C
Kelembapan : 74.10 %
=====

[TERIMA DARI] Node-3 (Node ID: 2743551631 | Chip ID: 2743551631)
Suhu : 30.60 °C
Kelembapan : 71.50 %
=====

[KIRIM MESH] {"node": "Node-2", "chipId": 3157789783, "suhu": 30.8, "kelembapan": 71.2}

```
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-3";
16
17 Scheduler userScheduler;
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM5')

Kelembapan : 71.50 %
=====

[TERIMA DARI] Node-1 (Node ID: 3158125985 | Chip ID: 3158125985)
Suhu : 30.60 °C
Kelembapan : 74.50 %
=====

[KIRIM MESH] {"node": "Node-3", "chipId": 2743551631, "suhu": 30.6, "kelembapan": 72.1}

[TERIMA DARI] Node-2 (Node ID: 3257789783 | Chip ID: 3257789783)
Suhu : 30.70 °C
Kelembapan : 71.50 %
=====

Ln 15, Col 31

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Simpul - Praktikum 2):

Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status "Berhasil Diterima" (`sendStatus == 0`), lakukan eksperimen pemutusan simpul:

1. Cabut kabel USB salah satu board Penerima (misal NodeMCU 2) dari sumber daya.
2. Amati perubahan log pada Serial Monitor NodeMCU Controller pengirim! Apakah status pengiriman untuk MAC address board yang dicabut berganti menjadi "Gagal (Tidak Terjangkau)"?

Jawaban:

Ya. Ketika kabel USB atau sumber daya salah satu board penerima dicabut, callback status pengiriman (`send_cb / OnDataSent`) pada NodeMCU Controller pengirim akan mendeteksi bahwa pengiriman paket gagal. Nilai `sendStatus` tidak lagi bernilai 0 (`ESP_NOW_SEND_SUCCESS`), melainkan mengembalikan status error (seperti `ESP_NOW_SEND_FAIL`), yang menandakan paket gagal terkirim ke MAC address tersebut.

3. Jelaskan bagaimana protokol ESP-NOW pada lapisan *Layer 2 MAC* dapat mendeteksi kegagalan transmisi ke perangkat fisik tersebut meskipun sistem tidak menggunakan koneksi TCP!

Jawaban:

Meskipun ESP-NOW bersifat *connectionless* dan tidak menggunakan protokol TCP/IP yang memiliki mekanisme *handshake* dan *acknowledgement* di layer transport, ESP-NOW beroperasi langsung pada **Layer 2 (Data Link Layer)** model OSI di atas bingkai (*frames*) Wi-Fi vendor Espressif.

1. Pada lapisan MAC 802.11, setiap pengiriman data *unicast* secara hardware mewajibkan adanya sinyal konfirmasi berupa **MAC ACK (Acknowledgment frame)** dari perangkat penerima.
2. Jika pengirim mengirim paket ke MAC address tertentu tetapi tidak menerima sinyal ACK dalam batas waktu (*timeout*) perangkat keras tertentu, pengendali Wi-Fi akan melakukan beberapa kali percobaan ulang (*retries*).

3. Jika semua percobaan ulang gagal karena perangkat fisik telah mati atau terputus, pengendali Wi-Fi menyatakan transmisi gagal dan meneruskan status kegagalan tersebut ke fungsi callback ESP-NOW di tingkat aplikasi.

Latihan 2 (Eksperimen Dinamika Topologi & Self-Healing - Praktikum 4):

Jalankan jaringan *painlessMesh* pada Praktikum 4 hingga seluruh board saling terhubung dan mencetak pesan --> Koneksi Baru Terdeteksi! Node ID:

- i. Tekan tombol fisik **RST** atau cabut daya salah satu board Node. Amati Serial Monitor pada board lainnya yang masih menyala. Berapa detik waktu yang dibutuhkan oleh *callback changedConnectionCallback()* untuk mendeteksi bahwa salah satu simpul telah keluar dari rantai mesh?

Jawaban:

Hasil Observasi Tipikal: Berdasarkan konfigurasi *heartbeat* bawaan *painlessMesh*, waktu yang dibutuhkan berkisar antara **5 hingga 15 detik**. Jaringan memerlukan beberapa siklus *node timeout* atau hilangnya paket *heartbeat* dari node yang bersangkutan untuk menyatakan bahwa node tersebut benar-benar terputus dari rantai mesh.

- ii. Sambungkan kembali daya board tersebut. Berapa lama waktu yang dibutuhkan hingga node tersebut kembali terdaftar secara otonom ke dalam topologi mesh tanpa Anda mengonfigurasi ulang jaringan? Catat hasil observasi waktu tersebut!

Jawaban:

Hasil Observasi Tipikal: Setelah daya disambungkan kembali, node akan melakukan proses *booting*, memindai (*scanning*) saluran Wi-Fi, dan bergabung kembali (*rejoin*) ke jaringan mesh dalam waktu sekitar **10 hingga 30 detik** secara otonom tanpa perlu mengatur ulang konfigurasi jaringan apa pun (berkat fitur *self-healing* topologi mesh).

Latihan 3 (Eksperimen Modifikasi Filter Telemetry & Ambang Batas JSON - Praktikum 4): Pada Praktikum 4, setiap node saat ini mencetak seluruh data telemetry dari semua pengirim tanpa penyaringan.

- i. Modifikasilah fungsi `receivedCallback` pada salah satu board agar bertindak sebagai **Pengawas Kondisi Suhu Kritis**: program **HANYA** mencetak kartu telemetry penuh ke Serial Monitor jika suhu yang dilaporkan oleh node tetangga bernilai **di atas 32.0 °C** (lakukan pengujian dengan mendekatkan jari hangat atau meniup sensor DHT pada node pengirim).
- ii. Jika suhu bernilai normal (≤ 32.0 °C), program cukup mencetak status singkat satu baris: `[NORMAL] Telemetry dari Node <chipId> aman`.
- iii. Tuliskan blok fungsi `receivedCallback(uint32_t from, String &msg)` hasil modifikasi Anda!

Jawaban:

```

1  #include <ArduinoJson.h>
2
3  void receivedCallback(uint32_t from, String &msg) {
4      // Alokasi dokumen JSON (sesuaikan ukuran memori jika diperlukan)
5      JsonDocument doc;
6      DeserializationError error = deserializeJson(doc, msg);
7
8      // Periksa apakah pesan berformat JSON yang valid
9      if (error) {
10         // Jika pesan bukan JSON, cetak apa adanya
11         Serial.printf("Pesan non-JSON dari Node %u: %s\n", from, msg.c_str());
12         return;
13     }
14
15     // Ambil nilai suhu dari JSON (sesuaikan key "suhu" dengan payload pengirim Anda)
16     float suhu = doc["suhu"] | 0.0;
17
18     if (suhu > 32.0) {
19         // Jika suhu di atas 32.0 °C, cetak kartu telemetry penuh
20         Serial.println("=====");
21         Serial.printf("[PERINGATAN] Suhu Kritis Terdeteksi!\n");
22         Serial.printf("Node ID   : %u\n", from);
23         Serial.printf("Suhu      : %.2f °C\n", suhu);
24         Serial.printf("Payload   : %s\n", msg.c_str());
25         Serial.println("=====");
26     } else {
27         // Jika suhu normal ( $\leq 32.0$  °C), cetak status singkat satu baris
28         Serial.printf("[NORMAL] Telemetry dari Node %u aman (%.2f °C).\n", from, suhu);
29     }
30 }
31

```